# Lesson 4: Direct Hit

Godot Engine

VIDEO GAME CREATION

This week: detecting when things touch. Your job is to add one enemy to the screen and make your bullet destroy it on contact - the first piece of your capstone's actual combat loop.

**Open your own [YourName]Game project from Lesson 3 - don't start a new one. Everything this week builds on the player and bullet you already have.**

*Tip: this lesson's Assets folder has a ready-to-use placeholder enemy image (enemy_invader_red.png) and a new sound effect (sfx_hit.wav) - you don't need to find or make your own art today.*

## Setup

1. Open your [YourName]Game project from Lesson 3.
2. Open bullet.tscn and add an Area2D child to the Bullet root, rename it Hitbox, then add a CollisionShape2D child to Hitbox with a new RectangleShape2D sized to roughly match bullet_yellow.png.
3. Create a new scene: a Sprite2D root node, rename it Enemy, and assign enemy_invader_red.png (from this lesson's Assets folder) as its Texture. Add an Area2D child, rename it Hitbox, then add a CollisionShape2D child to it with a new RectangleShape2D sized to roughly match enemy_invader_red.png.
4. Save this new scene as enemy.tscn. Attach a new script to the Enemy root, extends Sprite2D.
5. Open your main scene and drag an instance of enemy.tscn into it, positioning it near the top of the play area - Godot's origin (0,0) is the top-left corner, and Y increases downward, so "near the top" means a small positive Y (something like 50-150), not a negative one or a value based on the screen's center.

## Level 1 - Minimal (MVP)

*A single enemy sits on screen. When your bullet touches it, both the bullet and the enemy disappear. This is a complete finish for today - reaching this is enough to move on to next week.*

6. Connect Hitbox's area_entered signal to the Enemy so it can respond when something touches it - two ways to do this, pick one, not both: (a) in the editor, select Hitbox, open the Signals tab (grouped with Inspector and Groups), double-click area_entered, choose Enemy as the receiver in the dialog, and accept the suggested method name - Godot writes the function stub into Enemy's script for you; or (b) skip the editor entirely and just write $Hitbox.area_entered.connect(_on_hitbox_area_entered) yourself inside _ready()

(code below). Using both at once double-connects the signal, so the handler fires twice per hit.

7. Run the scene and shoot the Enemy to test - both the bullet and the Enemy should disappear on contact.

```
# enemy.tscn's script (attached to the Sprite2D root):
extends Sprite2D

func _ready():
    $Hitbox.area_entered.connect(_on_hitbox_area_entered)

func _on_hitbox_area_entered(area):
    area.get_parent().queue_free()
    queue_free()
```

## Level 2 - Complete (Alpha/Beta)

*The enemy is destroyed cleanly and exactly once, even if you fire several bullets in a row - no errors, no lingering references, and a bullet that misses still behaves exactly like it did last week.*

- Fire several bullets in a row, including some that miss - each hit destroys exactly one bullet and the Enemy, and any bullet that misses keeps flying and disappears off-screen exactly like it did in Lesson 3.
- queue_free() is safe to call more than once on the same node (a repeat call is simply ignored), so even the rare case of two bullets overlapping the Enemy on the very same frame doesn't cause an error - both bullets are destroyed and the Enemy is destroyed exactly once.

## Level 3 - Refined (Ready to Publish) - pick any that interest you

- Sound: add an AudioStreamPlayer2D node to the Enemy scene (named Hit), Stream set to sfx_hit.wav. Freeing the Enemy immediately would cut the sound off before it finishes, so hide the Enemy and turn off its Hitbox right away, play the sound, then await its finished signal before actually freeing the node.
- Visual: give the Enemy its own modulate color (distinct from the bullet's yellow) in its _ready(), so it clearly reads as a threat rather than another shot.
- Structure: everything about how the Enemy responds to being hit - destroying the bullet, hiding itself, playing a sound - lives inside enemy.tscn's own script. bullet.tscn's own script doesn't need to know the Enemy exists at all; it only needed a Hitbox added to it so the Enemy's Area2D could detect it.

## If Something's Wrong

- Nothing happens on contact: confirm both bullet.tscn and enemy.tscn actually have a CollisionShape2D with a real shape assigned inside their Hitbox (an Area2D with no shape can't detect anything), and that neither Hitbox's Monitoring property was left off.
- Godot reports an error on area.get_parent(): confirm the signal connected is Hitbox's own area_entered, not the Enemy Sprite2D root's - Sprite2D nodes don't have that signal at all.
- Enemy disappears immediately but the bullet lingers a frame or two: this is expected - queue_free() defers actual deletion to the end of the current frame, not instantly.

- Enemy doesn't appear anywhere on screen, even though it shows up in the Scene tree (or the Remote tab while the game is running): before checking the Texture, check Transform > Position in the Inspector. Godot's origin is the top-left corner with Y increasing downward, so a position guessed from a different convention (screen-center origin, Y-up) can land the Enemy well off the top or side of the visible viewport with no error at all. Reposition it directly in the 2D editor - drag it in the viewport, or edit Position in the Inspector - rather than repositioning it from code in _ready(). That's the preferred fix here, not just a shortcut: it keeps what you see in the editor exactly matching your game's actual start position, the same way Player's position already works.
- Enemy's hit response seems to run twice (bullet and Enemy both vanish correctly, but a later Level 3 sound plays twice, or a score/lives change in a future lesson is off by double): the signal got connected two ways at once - both through the Signals tab in the editor and through a $Hitbox.area_entered.connect() call in _ready(). Select Hitbox and check its Signals tab; if area_entered already lists a connection to Enemy there, remove the code call (or vice versa) so only one connection exists.

## Want More?

Student Resource List > Godot section, or docs.godotengine.org